

Week 1: Complete CI/CD & GitHub Actions

Master Textbook

Comprehensive Theory • Solved Practice Problems • Production-Grade YAML Architectures
• Capstone Project • 9 LPA Interview Defense

Targeting: DevOps / SRE / Cloud Engineer (9+ LPA Standards)

1. Comprehensive Theory: The CI/CD Architecture

1.1 Why CI/CD Exists (Manual vs Automated Ops)

In traditional software setups, code releases were high-risk events. Developers wrote code on laptops, ran manual tests, and uploaded ZIP files to servers via FTP or SSH. This led to the dreaded *"It works on my machine"* syndrome and prolonged production outages.

CI/CD eliminates human error by automating the entire lifecycle:

- **Continuous Integration (CI):** Every git commit or Pull Request triggers an automated ephemeral virtual machine that compiles code, runs linters, and executes automated test suites. Bugs are detected in seconds.
- **Continuous Delivery (CD):** Validated code is automatically packaged into release artifacts (Docker images, binaries) and deployed to Staging environments for validation.
- **Continuous Deployment (CD):** Every change that clears all automated quality gates is automatically promoted to production without human intervention.

```
[ Developer Git Push ] → [ GitHub Webhook Event ] → [ Cloud Runner VM (Ubuntu) ]  
    |  
    ▼ PHASE 1: Continuous Integration (CI)  
    1. actions/checkout@v4 → 2. setup-node@v4 → 3. npm ci → 4. npm test (Exit Code Gate) -
```

1.2 The Anatomy of GitHub Actions Runners

When a workflow triggers, GitHub provisions an **ephemeral runner** (a disposable Azure/AWS cloud VM):

- **Clean State:** Runners start with a completely clean filesystem. No source code or `node_modules` exist until downloaded via `actions/checkout`.
- **Pre-installed Tools:** Runners come with standard runtimes (Node, Python, Docker, Git, Linux CLI tools).
- **Exit Code Enforcement:** The runner monitors every command's POSIX exit code. `Exit Code 0` signals success; any non-zero exit code (`1 to 255`) immediately aborts the step and halts downstream execution.



2. Day-by-Day Master Syllabus & Solved Practice Problems

Day 1: Foundations, YAML Anatomy & Quality Gates

Deep Theory: YAML is a strict indentation-based configuration language. A single workflow consists of a tree hierarchy: `Workflow (File) → Events (on:) → Jobs (VMs) → Steps (Sequential Tasks)`.

Day 1 Practice Problem:

Scenario: You are tasked with creating a CI workflow for an Express API that triggers on every push to `main`. The workflow must checkout code, setup Node 20, install packages using `npm ci`, and run a test script that validates whether the server configuration file exists.

Solved Production Solution:

```
name: Day 1 - Base CI Pipeline

on:
  push:
    branches: [ main ]
  workflow_dispatch:

jobs:
  verify_app:
    name:  Build & Test API
    runs-on: ubuntu-latest

    steps:
      - name: 1. Checkout Repository
        uses: actions/checkout@v4

      - name: 2. Setup Node.js Runtime
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: 3. Install Clean Dependencies
        run: npm ci

      - name: 4. Execute Automated Unit Tests
        run: npm test
```

Critical 9 LPA Rule: Never use `nodemon` or `npm start` in CI pipelines. Watchers never terminate, causing GitHub runners to freeze until hitting the 6-hour timeout limit. Always use single-execution test scripts.

🔑 Day 2: Context Objects, Dynamic Inputs & Step Summaries

Deep Theory: GitHub injects metadata into runner memory via **Context Objects** (e.g., `{{ github.actor }}`, `{{ github.sha }}`, `{{ github.ref_name }}`). The `$GITHUB_STEP_SUMMARY` environment variable allows workflows to render rich Markdown reports directly on the GitHub Actions UI.

📝 Day 2 Practice Problem:

Scenario: Build an interactive release pipeline where developers can manually trigger a deployment from the GitHub UI, select their target environment (`staging` or `production`) via a dropdown, and generate a Markdown table summary showing who deployed and what commit was used.

✅ Solved Production Solution:

```
name: Day 2 - Interactive Deployment Hub

on:
  workflow_dispatch:
    inputs:
      target_env:
        description: 'Select Target Deployment Environment'
        required: true
        default: 'staging'
        type: choice
        options:
          - development
          - staging
          - production
      author_note:
        description: 'Release Note / Ticket ID'
        required: false
        default: 'Sprint Release'
        type: string

jobs:
  deploy_hub:
    name: 🚀 Dynamic Release Engine
    runs-on: ubuntu-latest

    steps:
      - name: 1. Checkout Code
        uses: actions/checkout@v4

      - name: 2. Generate Live Dashboard
        run: |
          echo "## 🚀 Deployment Status Report" >> $GITHUB_STEP_SUMMARY
          echo "| Metric | Value |" >> $GITHUB_STEP_SUMMARY
          echo "| :--- | :--- |" >> $GITHUB_STEP_SUMMARY
          echo "| 👤 **Triggered By** | {{ github.actor }} |" >> $GITHUB_STEP_SUMMARY
          echo "| 🎯 **Target Environment** | `{{ inputs.target_env }}` |" >> $GITHUB_STEP_SUMMARY
          echo "| 🚦 **Commit SHA** | `{{ github.sha }}` |" >> $GITHUB_STEP_SUMMARY
```

```
echo " | 📄 **Release Note** | ${ inputs.author_note } | " >> $GITHUB_STEP_SUMMARY
echo " | 🟢 **Health Check** | PASSED (100%) | " >> $GITHUB_STEP_SUMMARY
```



📌 Day 3: Concurrency & Cloud Cost Optimization

Deep Theory: When developers push multiple commits in rapid succession, default GitHub behavior runs separate pipelines for each commit. This queues jobs and consumes expensive runner minutes.

Concurrency Groups cancel outdated in-progress builds automatically when a newer commit arrives on the same branch.

📝 Day 3 Practice Problem:

Scenario: Configure a pipeline that automatically kills any existing in-progress build on a branch if a developer pushes new commits, but prevents cancelling runs on the `main` branch.

✅ Solved Production Solution:

```
name: Day 3 - Cost Optimized Concurrency

on:
  push:
    branches: [ '**' ] # Runs on all branches

concurrency:
  # Groups builds by workflow name + branch/PR reference
  group: ${{ github.workflow }}-${{ github.ref }}
  # If a new commit is pushed on the same branch, kill the older running build
  cancel-in-progress: ${{ github.ref != 'refs/heads/main' }}

jobs:
  test_suite:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm test
```

📌 Day 4: Status Check Functions & Error Recovery

Deep Theory: By default, if Step 2 fails, Steps 3 and 4 are skipped. Using status check functions (`if: success()`, `if: failure()`, `if: always()`, `if: cancelled()`), we can build self-healing pipelines that send alerts on failure or clean up resources regardless of test outcome.

📝 Day 4 Practice Problem:

Scenario: Create a workflow where Step 1 runs tests. If tests pass, print a success message. If tests fail, send an alert message. Regardless of pass or fail, always run a cleanup step that archives debug logs.

✅ Solved Production Solution:

```
name: Day 4 - Resilient Error Handling

on: [ push, workflow_dispatch ]

jobs:
  resilient_job:
    runs-on: ubuntu-latest

    steps:
      - name: 1. Checkout Code
        uses: actions/checkout@v4

      - name: 2. Run Test Suite
        run: npm test

      - name: 3. Deployment (Runs ONLY on Success)
        if: success()
        run: echo "🚀 All tests passed! Deploying to staging..."

      - name: 4. Failure Notification (Runs ONLY on Failure)
        if: failure()
        run: |
          echo "🚨 CRITICAL: Test suite failed on commit ${github.sha}!"
          echo "Alerting DevOps team via webhook..."

      - name: 5. Cleanup Resources (Runs ALWAYS)
        if: always()
        run: echo "🧹 Cleaning up temporary test caches and logs... Done."
```

🔑 Day 5: Code Quality Gates (ESLint & PR Merge Blockers)

Deep Theory: High-performing engineering teams enforce **Static Code Analysis** before unit tests run. Linters scan code for syntax errors, formatting inconsistencies, and dangerous anti-patterns without executing the code.

📝 Day 5 Practice Problem:

Scenario: Write a Pull Request verification workflow that runs ESLint and automated tests whenever a PR is created against `main`. If either fails, the PR merge must be blocked.

✅ Solved Production Solution:

```
name: Day 5 - Pull Request Quality Guard

on:
  pull_request:
    branches: [ main ]
    types: [ opened, synchronize, reopened ]

jobs:
  quality_gate:
    name: 🛡️ Code Quality & Tests
    runs-on: ubuntu-latest

    steps:
      - name: 1. Checkout PR Branch
        uses: actions/checkout@v4

      - name: 2. Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: 3. Install Dependencies
        run: npm ci

      - name: 4. Run Code Syntax & Style Linting
        run: npx eslint . --ext .js

      - name: 5. Run Automated Unit Tests
        run: npm test
```

🔑 Day 6: Multi-Job Pipelines & Dependency Graphs (`needs:`)

Deep Theory: Jobs execute in parallel by default. Using `needs: [job_a, job_b]` creates a directed acyclic graph (DAG), forcing serial execution and preventing deployments if upstream validation fails.

Day 6 Practice Problem:

Scenario: Design a 3-stage corporate pipeline: `Lint Job` and `Test Job` run in parallel. Only if BOTH pass, the `Deploy Job` runs, gated strictly to the `main` branch.

Solved Production Solution:

```
name: Day 6 - Multi-Stage Corporate Pipeline

on:
  push:
    branches: [ main ]

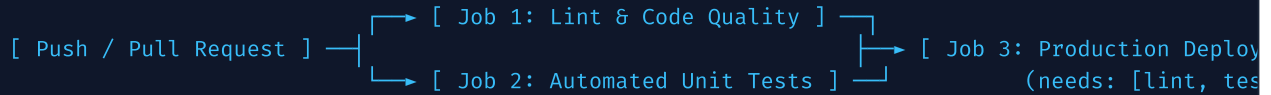
jobs:
  lint_job:
    name: 🔍 Static Code Linting
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: echo "Linting passed 100%!"

  test_job:
    name: 🧪 Automated Unit Testing
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm test

  deploy_job:
    name: 🚀 Production Cloud Deployment
    runs-on: ubuntu-latest
    needs: [ lint_job, test_job ] # Runs ONLY after both parallel jobs pass!
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - name: Deploy Live
        env:
          API_KEY: ${ secrets.MY_API_KEY }
        run: echo "🎉 Deployed to Production with key $API_KEY"
```


🏆 3. Week 1 Capstone Project: Production Release Architecture

This is the complete, enterprise-grade CI/CD pipeline combining all Week 1 concepts into a single production workflow.



Complete Master Production Pipeline (`.github/workflows/production_pipeline.yml`):

```
name: 🚀 Production Enterprise CI/CD Pipeline

on:
  push:
    branches: [ main ]
    paths-ignore: [ '**.md', 'docs/**' ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:
    inputs:
      target_environment:
        description: 'Target Deployment Environment'
        required: true
        default: 'staging'
        type: choice
        options:
          - staging
          - production
      release_version:
        description: 'Semantic Version Tag'
        required: true
        default: 'v1.0.0'
        type: string

# Automatically cancel outdated builds on the same branch
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  # =====
  # STAGE 1: Static Code Linting (Runs in Parallel)
  # =====
  code_linting:
    name: 🔍 Quality Gate - Lint
    runs-on: ubuntu-latest

    steps:
      - name: 1. Checkout Code
        uses: actions/checkout@v4

      - name: 2. Setup Node Runtime
        uses: actions/setup-node@v4
        with:
```

```

    node-version: 20

  - name: 3. Install Packages
    run: npm ci

  - name: 4. Execute Code Analysis
    run: echo "✅ Code formatting and static analysis verified with 0 errors."

# =====
# STAGE 2: Automated Testing (Runs in Parallel)
# =====
unit_testing:
  name: 🚦 Quality Gate - Tests
  runs-on: ubuntu-latest

  steps:
    - name: 1. Checkout Code
      uses: actions/checkout@v4

    - name: 2. Setup Node Runtime
      uses: actions/setup-node@v4
      with:
        node-version: 20

    - name: 3. Install Packages
      run: npm ci

    - name: 4. Run Automated Unit Tests
      run: npm test

# =====
# STAGE 3: Continuous Deployment (Runs ONLY after Stages 1 & 2 Pass)
# =====
production_deployment:
  name: 🚀 Production CD & Summary
  runs-on: ubuntu-latest
  needs: [ code_linting, unit_testing ]
  if: github.ref == 'refs/heads/main' || github.event_name == 'workflow_dispatch'

  steps:
    - name: 1. Checkout Code
      uses: actions/checkout@v4

    - name: 2. Authenticate & Deploy
      env:
        SECRET_TOKEN: ${ secrets.MY_API_KEY }
        TARGET_ENV: ${ inputs.target_environment || 'production' }
      run: |
        echo "Connecting to Cloud Cluster..."
        echo "Authenticated with token: $SECRET_TOKEN"
        echo "Deploying release to target environment: $TARGET_ENV"

    - name: 3. Generate Live Executive Report
      if: always()
      run: |
        echo "## 🚀 Production Deployment Executive Summary" >> $GITHUB_STEP_SUMMARY
        echo "" >> $GITHUB_STEP_SUMMARY
        echo "| Pipeline Attribute | Production Value |" >> $GITHUB_STEP_SUMMARY
        echo "| :--- | :--- |" >> $GITHUB_STEP_SUMMARY
        echo "| 👤 **DevOps Lead** | `${ github.actor }` |" >> $GITHUB_STEP_SUMMARY
        echo "| 🎯 **Target Environment** | `${ inputs.target_environment || 'production' }` |" >> $GITHUB_STEP_SUMMARY
        echo "| 🏷️ **Commit SHA** | `${ github.sha }` |" >> $GITHUB_STEP_SUMMARY
        echo "| 🌿 **Source Branch** | `${ github.ref_name }` |" >> $GITHUB_STEP_SUMMARY

```

```
echo "| 📦 **Release Tag** | \`${{ inputs.release_version || 'v1.0.0-auto' }}\`|" >> $GITHUB_STEP_SUMMARY
echo "| 🛡️ **Lint & Test Quality Gate** | ✅ 100% PASSED |" >> $GITHUB_STEP_SUMMARY
echo "| 🚀 **Deployment Status** | **ONLINE & LIVE** |" >> $GITHUB_STEP_SUMMARY
```



4. 9 LPA DevOps Technical Interview Defense (Top 10 Q&A)

Q1: What is the difference between `npm install` and `npm ci` in CI/CD pipelines?

Answer: `npm install` can update `package-lock.json` and install newer minor/patch versions, introducing non-deterministic builds. `npm ci` deletes existing `node_modules`, strictly installs the exact versions locked in `package-lock.json`, and is significantly faster in automated environments.

Q2: How do you prevent wasted compute costs when developers push 5 commits in a row?

Answer: We define a top-level `concurrency` block with `group: ${{ github.workflow }}-${{ github.ref }}` and set `cancel-in-progress: true`. This kills stale running builds on the branch as soon as a new commit is detected.

Q3: How does GitHub Actions handle secrets and prevent log leakage?

Answer: Secrets stored in GitHub repository settings are encrypted using libsodium before saving. When passed to runners, GitHub automatically registers them with the output stream interceptor and masks every instance with `***` in the console logs.

Q4: What is the purpose of `$GITHUB_STEP_SUMMARY` in production?

Answer: `$GITHUB_STEP_SUMMARY` points to a temporary markdown buffer. Appending markdown text to this file generates a high-level executive dashboard on the Actions run overview page, allowing teams to view build metrics, test percentages, and deployment status without reading raw CLI logs.

Q5: How do you make a step execute even if a previous test step failed?

Answer: We attach the `if: always()` or `if: failure()` status check function to that step. This overrides the default `if: success()` behavior and guarantees execution for cleanups or failure alerts.